REINFORCEMENT LEARNING FOR SELF-VERIFYING CODE GENERATION

Justin Medich
Department of Computer Science
University of Texas at Austin
jmedich@utexas.edu

ABSTRACT

Large language models can generate syntactically correct code yet frequently fail in ways that reflect shallow or inconsistent reasoning. Most systems rely on external unit tests to determine correctness, leaving models without mechanisms to verify their own logic. This project investigates whether reinforcement learning can improve code reliability by training a model to produce not only program implementations, but also self-generated tests and predicted execution traces that reflect its internal reasoning. By rewarding consistency between these self-verifications and ground-truth program behavior, the approach aims to align a model's reasoning steps with executable truth. Using an augmented version of the MBPP dataset and a PPO-based training pipeline, the method demonstrates improvements in both code accuracy and reasoning coherence over a supervised baseline. The results suggest that verifiable process-level signals offer a promising path toward more trustworthy, self-checking code generation models.

1 BACKGROUND AND MOTIVATION

Large language models have become increasingly proficient at generating executable code, achieving strong performance on benchmarks such as MBPP and HumanEval. Yet despite steady improvements in syntactic fluency and pattern recognition, these systems frequently fail in ways that reveal that they lack reliable mechanisms for verifying their own reasoning. Even when code looks right, the underlying logic may be incorrect, and most current approaches depend entirely on externally provided unit tests to detect such failures. As a result, models often produce plausible but flawed solutions that pass shallow heuristics but fail under careful execution.

At the same time, recent progress in reasoning-focused language modeling has demonstrated that supervising or rewarding intermediate reasoning steps, rather than just final outcomes, can improve both accuracy and consistency. Methods such as process reward models, chain-of-thought verification, and self-refinement illustrate that models benefit from being trained not only to produce answers, but also to examine and justify the steps leading to those answers. However, these advances have largely centered on mathematical or symbolic reasoning, leaving open the question of how similar ideas might be extended to executable domains like program synthesis.

Instead of treating program correctness as a binary, outcome-only signal, my project introduces a reinforcement learning framework that encourages models to generate self-verifying structure alongside codew with their own unit tests and their own predicted execution traces. The core contribution is showing that giving models the incentive to construct and align these internal, verifiable reasoning artifacts can improve their reliability on code generation tasks. The approach reframes code synthesis as not just producing a program, but producing a coherent explanation of how the program behaves—and then rewarding consistency between that explanation and the ground truth.

This perspective positions self-verifying code generation as a promising direction for bridging the gap between high-level reasoning rewards and execution-based correctness. By encouraging models to articulate and validate their own logic, the work moves toward systems that fail more transparently, reason more consistently, and ultimately generate code that better reflects the true behavior of the underlying algorithms they are meant to express.

2 RELATED WORK

2.1 EXECUTION-BASED REINFORCEMENT LEARNING FOR CODE

Early work in program synthesis with large language models has shown that execution feedback can substantially improve correctness. Chen et al. Chen et al. (2021) demonstrated that evaluating generated code against unit tests provides stronger and more actionable signals than static metrics such as BLEU. CodeRL Le et al. (2022) further formalized this idea with an actor–critic architecture that rewards programs for passing tests, achieving gains on benchmarks including MBPP and HumanEval. Subsequent methods such as Reflexion Shinn et al. (2023) and Self-Refine Madaan et al. (2023) explored iterative self-correction based on failed test cases. However, these approaches depend entirely on externally provided tests and do not supervise or reward the model's internal reasoning processes.

2.2 PROCESS-LEVEL REWARDS FOR REASONING

Recent advances in reasoning-focused language modeling emphasize supervising intermediate steps rather than evaluating solutions solely by final correctness. Process supervision, as introduced in Wang et al. Wang et al. (2023a), and step-verification approaches such as "Let's Verify Step by Step" Lightman et al. (2023), show that rewarding consistency within a chain of reasoning leads to more reliable outputs in arithmetic and symbolic tasks. These methods illustrate the benefit of aligning model behavior with verifiable reasoning traces, but they have been applied almost exclusively to natural language or mathematical reasoning, leaving their applicability to executable domains largely unexplored.

2.3 EXPLANATION AND INTERPRETABILITY IN CODE GENERATION

Several efforts have aimed to make code generation models more interpretable by coupling programs with natural-language rationales. CodeExplain Yuan et al. (2022) provides paired explanations to encourage faithful reasoning, while CodeT5+ Wang et al. (2023b) uses rationale-augmented fine-tuning to improve explainability and code understanding. Although these methods enhance interpretability, they rely on linguistic supervision rather than executable validation. They do not require models to produce reasoning artifacts that can be checked against the true behavior of a program.

2.4 GAPS IN VERIFIABLE REASONING FOR CODE

Despite growing interest in both execution-based feedback and process-level supervision, no existing framework combines these directions by training models to generate verifiable, intermediate reasoning artifacts, such as predicted execution traces or self-generated tests, and rewarding their alignment with ground-truth behavior. This gap motivates my present work, which explores whether reinforcement learning with verifiable, execution-level signals can improve the consistency, accuracy, and self-verification capabilities of code generation models.

3 METHODOLOGY AND IMPLEMENTATION

3.1 DATASET AUGMENTATION

The method begins by extending the MBPP dataset to include structured, verifiable reasoning signals. For each problem, the reference solution is executed under a Python tracing instrument, which records:

- **Gold execution traces**: line-level state transitions, function calls, variable assignments, and return values.
- **Gold input–output mappings**: derived from running the official MBPP test cases, providing consistent behavioral ground truth.
- **Structured target format**: each training instance contains three supervised segments,

```
<CODE> ... </CODE>
<TESTS> ... </TESTS>
<TRACE> ... </TRACE>
```

enabling the model to learn to output code, self-generated tests, and predicted traces jointly.

This augmentation transforms MBPP into a dataset suitable for process-level supervision, where runtime behavior becomes an explicit learning signal.

3.2 BASE MODEL: CODET5+

The implementation in this work uses **CodeT5+ 770M-Py** as the base model for both supervised fine-tuning and reinforcement learning. CodeT5+ is an extension of the original CodeT5 architecture, incorporating improvements in code understanding, multi-task training, and rationale-augmented learning. The 770M-Py variant is specifically pretrained on large-scale Python corpora, making it well suited for the MBPP-style synthesis tasks considered in this project.

CodeT5+ follows an encoder–decoder transformer architecture, enabling it to generate structured, multi-segment outputs such as the required `<CODE>`, `<TESTS>`, and `<TRACE>` blocks. Compared to decoder-only code models, the encoder–decoder structure has two advantages in this setting:

- **Stronger conditioning on the prompt:** The encoder processes the natural-language problem description, enabling more grounded generation during decoding.
- **Flexibility in structured generation:** Because the decoder is trained to output multiple distinct segments, the architecture handles long, multi-part outputs more reliably than typical decoder-only LLMs.

The 770M-Py model is also computationally feasible for PPO fine-tuning: its size lies in a practical middle ground, large enough to capture semantic and syntactic patterns in Python, yet small enough to allow repeated rollouts, code execution, and gradient updates during reinforcement learning.

During supervised fine-tuning, the model learns to generate all three structured segments in a single pass. During PPO training, the model is wrapped using `AutoModelForSeq2SeqLMWithValueHead`, enabling the integration of a value function for token-level advantage estimation.

3.3 SUPERVISED FINE-TUNING

Before reinforcement learning, the model undergoes supervised fine-tuning (SFT) on the augmented dataset. The SFT objective is a standard next-token cross-entropy loss over the concatenated `<CODE>`, `<TESTS>`, and `<TRACE>` blocks. This stage serves two purposes:

1. It encourages the model to reliably produce the required output structure, reducing malformed generations during RL.
2. It provides an initial approximation of execution trace prediction, improving reward stability when RL begins.

3.4 REINFORCEMENT LEARNING FRAMEWORK

After SFT, the model is trained with Proximal Policy Optimization (PPO) using a composite reward designed to align code correctness with internal reasoning consistency. The implementation follows the TRL framework, using a sequence-to-sequence backbone augmented with a value head.

During each RL iteration, the model generates a candidate output. Regular expressions extract the contents of the `<CODE>`, `<TESTS>`, and `<TRACE>` segments, and incorrectly formed or missing segments gain structural penalties.

3.5 EXECUTION SANDBOX AND TRACING

A Python sandbox executes the generated code and evaluates the self-generated tests. The sandbox performs three key tasks:

1. **Code execution**: the <CODE> block is run in an isolated namespace.
2. **Self-test evaluation**: the <TESTS> block is executed against the generated program to produce a pass/fail signal (A series of asserts).
3. **Trace collection**: a custom tracer logs line events, function calls, local variable states, and return values.

To maintain stability and prevent runaway logs (due to multiple recursion based problems), the tracer caps execution at 100 function calls. The resulting execution trace becomes the basis for computing alignment with the model-generated <TRACE> block.

TRACE ALIGNMENT COMPUTATION

To evaluate whether the model's predicted execution trace accurately reflects the true runtime behavior of the program, the system computes a trace-alignment score between the generated <TRACE> block and the ground-truth trace collected by the sandbox tracer. Both traces are first converted into ordered lists of *execution steps*, where each step corresponds to a structured event such as:

- a function call (e.g., `call sum_average`),
- a line execution event (e.g., `line sum_average:3`),
- a state snapshot showing local variable values.

The alignment score is then computed by iterating through the generated trace and counting how many of its steps exactly match the corresponding steps in the ground-truth sequence. A match is recorded only when the event type, associated function or line number, and (when applicable) the local-variable dictionary all agree. Let G denote the ordered list of generated steps and T the ordered list of ground-truth steps. The trace-alignment score is defined as:

$$R_{\text{trace}} = \frac{|\{\, i \mid G_i = T_i \,\}|}{|T|}.$$

This quantity measures the proportion of ground-truth execution events that the model correctly predicts in both content and order. The score naturally lies in $[0, 1]$ and rewards the model for producing a trace that mirrors the true sequence of runtime actions. Because execution traces are highly sensitive to small syntactic or logical changes in code, and because even one deviation can shift subsequent steps, this component tends to improve more slowly and noisily than other reward terms. Nonetheless, it provides a valuable process-level signal that encourages the model to internalize the structure of program execution rather than optimizing solely for passing tests.

TEST-FORMATION VERIFICATION

To ensure that the model's self-generated tests are syntactically meaningful and target the intended function, each test case is validated against the generated function header. After extracting the function name from the <CODE> segment using a regular expression (e.g., matching the pattern `def <name>(...):`), the system checks that every test follows the expected assertion structure. A secondary regex verifies that tests conform to the format

```
assert <func>(...)  == <value>
```

3.6 REWARD DESIGN

The reinforcement learning objective uses a composite reward of the form:

$$R = \alpha R_{\text{code}} + \beta R_{\text{tests}} + \gamma R_{\text{trace}}, \tag{1}$$

where R_{code} measures whether the program executes correctly on ground truth tests, R_{tests} captures the proportion of passing self-generated tests, and R_{trace} evaluates similarity between predicted and observed execution traces. In practice, the reward is broken down into stable increments:

- **Formatting reward (+0.1)**: awarded for producing well-formed <CODE>, <TESTS>, and <TRACE> blocks.
- **Execution reward (+0.3)**: given when the program runs without errors on ground truth tests.
- **Test reward (+0.4)**: proportional to the number of passing self-tests.
- **Trace reward (up to +0.2)**: based on normalized alignment between predicted and true traces.

Missing, malformed, or contradictory segments incur penalties. This design encourages the model to produce not only correct programs, but also coherent and verifiable reasoning traces.

3.7 Pipeline Summary

The overall system proceeds in four stages shown in Figure 1:

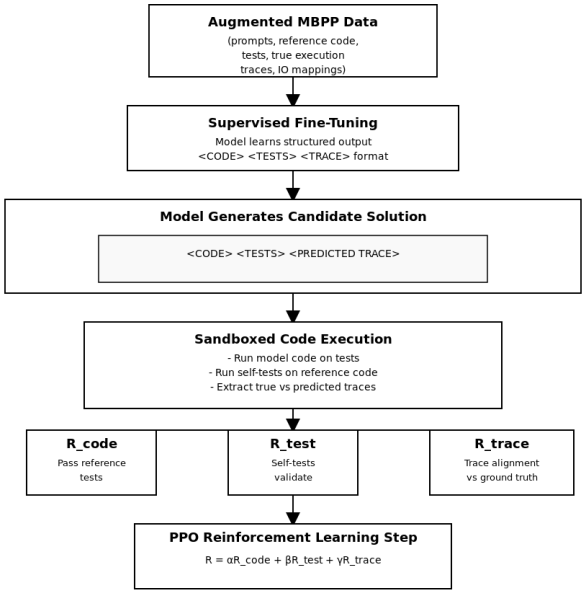


Figure 1: Overview of Training Pipeline.

1. Augment MBPP with execution traces and structured targets.
2. Train a supervised model to generate <CODE>, <TESTS>, and <TRACE> blocks.
3. Execute generated code in a sandbox and compare predicted traces with actual behavior.
4. Use PPO to optimize the model with a composite reward function based on sandbox executions.

4 Experiments and Results

This section summarizes the experimental setup and the results obtained from evaluating the supervised baseline and the reinforcement learning (RL) model on a held-out subset of MBPP. The evaluation focuses on two questions: (1) whether reinforcement learning with self-verifying rewards improves code correctness, and (2) whether models learn to generate more coherent tests and execution traces that better align with ground-truth behavior.

4.1 EXPERIMENTAL SETUP

A validation set of 100 MBPP problems was sampled from the training distribution. The baseline model and the RL-trained model were each evaluated on this identical set. All evaluations were performed using *Pass@1*: a solution is counted as correct only if the first generated program passes all official test cases.

For the RL model, each PPO iteration involved: generating model outputs, parsing the <CODE>, <TESTS>, and <TRACE> segments, executing the resulting code in a sandbox, and computing a composite reward. Training proceeded until convergence in the mean reward curve, which empirically stabilized after several thousand updates.

4.2 QUANTITATIVE RESULTS

Figure 2 (as shown in the presentation) reports the Pass@1 accuracy of both systems on the 100-problem validation set. The baseline achieved a Pass@1 accuracy of 25%, while the RL-trained model improved to 36%. This represents an absolute improvement of 11 percentage points and a relative gain of 44%.

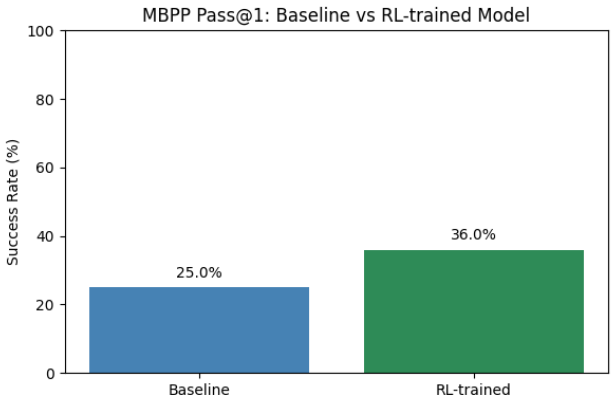


Figure 2: Comparison of Pass@1 accuracy between the baseline and the RL-trained model.

Figure 3. reveals how different aspects of model behavior improve at distinct stages of PPO training. Format correctness rises rapidly and plateaus early, indicating that the model quickly masters the structural requirements of producing well-formed <CODE>, <TESTS>, and <TRACE> blocks. In contrast, the rate at which generated code executes successfully improves more gradually, reflecting the greater difficulty of producing syntactically valid and semantically stable Python programs. Self-test correctness increases later in training, largely following the rise in code executability, suggesting that meaningful test generation depends on the model first producing runnable code. Finally, trace-alignment accuracy exhibits the slowest and noisiest improvement, highlighting that predicting detailed runtime behavior is the most challenging component of the task. Together, the curves show a natural learning progression: PPO first stabilizes surface structure, then improves program validity, and only later enhances the deeper reasoning signals represented by tests and traces.

4.3 TRACE AND TEST QUALITY

The RL-trained model also produced clearer, more meaningful <TESTS> and <TRACE> segments than the SFT-only model. Self-tests increasingly resembled human-written test cases, often covering edge cases involving empty strings, small integers, or boundary conditions. Trace predictions similarly became more aligned with actual runtime behavior, with fewer hallucinated function calls or missing state transitions. Although perfect trace alignment remained challenging, the overall trend showed a reduction in noise and an improvement in logical coherence.



Figure 3: Individual reward components over training.

4.4 Summary of Findings

Overall, the experiments demonstrate that combining execution-based feedback with process-level supervision yields measurable improvements in both correctness and reasoning quality. The RL model not only passes more tests but also produces outputs whose internal reasoning more closely match true program behavior. These results support the hypothesis that verifiable intermediate signals can enhance the reliability of code generation models.

5 Discussion and Limitations

The empirical results indicate that integrating process-level verification signals into reinforcement learning can meaningfully improve the reliability of code generation. The RL-trained model produces code that is not only more correct on MBPP-style tasks, but also supported by clearer self-generated tests and more coherent predicted execution traces. These gains suggest that rewarding models for articulating and validating their own reasoning steps encourages deeper semantic alignment than outcome-only training. In particular, the improvements in trace quality indicate that the model is learning aspects of program execution structure, rather than merely optimizing for surface-level syntactic patterns. This points toward the broader potential of verifiable intermediate signals as a tool for guiding model reasoning in executable domains.

Despite these promising outcomes, several limitations highlight opportunities for future work. First, the execution-and-tracing pipeline introduces substantial computational overhead: running each candidate program in a sandbox, generating a full execution trace, and evaluating self-tests significantly increases training time compared to standard PPO setups. This limits the scale of feasible experiments and constrains the size of models that can be effectively trained. Second, trace alignment remains a challenging objective. Because execution traces are sensitive to small syntactic or logical variations, the alignment score can be noisy, and even minor deviations may cause a cascade of mismatched steps. More robust or hierarchical trace representations may help stabilize this component of the reward. Third, MBPP is relatively small, and it remains unclear how well the method generalizes to larger, multi-file codebases or tasks requiring more complex reasoning. Finally, the approach focuses on single-shot generation; incorporating iterative refinement or multi-turn self-verification may further strengthen the model's ability to reason about and debug its own outputs.

Addressing these limitations, particularly the computational cost of execution-based RL, the fragility of trace alignment, and the need for larger and more diverse training corpora—represents an important direction for future work toward more scalable, trustworthy self-verifying code models.

REFERENCES

Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Hung Le et al. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *NeurIPS*, 2022.

Jacob Lightman et al. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050*, 2023.

Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

Noah Shinn et al. Reflexion: An autonomous agent with dynamic memory and self-reflection. *arXiv preprint arXiv:2303.11366*, 2023.

Xuezhi Wang et al. Process supervision improves reasoning in large language models. *arXiv preprint arXiv:2305.20050*, 2023a.

Yue Wang et al. CodeT5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*, 2023b.

Huiyuan Yuan et al. Codeexplain: A dataset for explainable code generation. *arXiv preprint arXiv:2204.12920*, 2022.